

Chapter 1

Day 1

1.1 Motivation

Before covering MARL, we need to cover RL first, and before that dynamic programming (DP).

Why DP? Thus far we know how to approach **deterministic problems**. When working with discrete states and discrete time (Grid-like problems) we can employ methods like A*. For Continuous state problems, we can use Model Predictive Control (MPC), where we solve:

$$\min \sum_{t=0}^T \text{cost}(x_t, u_t)$$

(subject to dynamics and constraints)

In both these cases, we have deterministic dynamics defined by:

$$x_{t+1} = f(x_t, u_t)$$

If I take action u_t in state x_t , I will end up at state x_{t+1} .

But what if outcomes are stochastic?

- With probability $P = 0.9$, I reach state x'_{t+1}
- With probability $P = 0.1$, I reach a different state x''_{t+1}

1.2 Stochasticity (Nature)

We now have probabilistic transitions, where moving from one state to another is probabilistic. This randomness is described as the influence from *Nature*, represented through a set of external, uncontrollable variables.

Let $\Theta(x_t^i, u_t^i)$ be the set of Nature's possible actions θ_t^i given state x_t^i and action u_t^i .

- These outcomes can be probabilistic, or worst-case.
- The specific $\theta_t^i \in \Theta$ is unknown from the agent's perspective.

- Assume a Markov property for Nature's actions:

$$P(\theta_t^i \mid o_{1:t}^i, u_{1:t}^i) = P(\theta_t^i \mid x_t^i, u_t^i)$$

This means that the distribution over Nature's actions at time t depends only on the current state x_t^i and current action u_t^i , not on the full history of past observations $o_{1:t}^i$ or actions $u_{1:t}^i$. In other words, Nature's behavior has no memory and is only conditioned on the present

The state transition is now dependent on Nature's action:

$$x_{t+1}^i = f(x_t^i, u_t^i, \theta_t^i)$$

Define the reachable set of next states under all possible Nature actions:

$$x_{t+1}^i = \{x \in \mathcal{X} \mid \exists \theta_t^i \in \Theta(x_t^i, u_t^i) \text{ such that } x = f(x_t^i, u_t^i, \theta_t^i)\}$$

In the probabilistic case, the transition probability is defined by marginalizing over all possible values of θ_t^i :

$$P(x_{t+1}^i \mid x_t^i, u_t^i) = \sum_{\theta_t^i \in \Theta(x_t^i, u_t^i)} P(\theta_t^i \mid x_t^i, u_t^i)$$

We can define the set of Nature's actions $\Theta(x_t^i, u_t^i)$ for a given state-action pair (x_t^i, u_t^i) .

$$\Theta(x_t^i, u_t^i) = \{\theta_t^i \mid \delta(\Theta(x_t^i, u_t^i)) \text{ and } x_{t+1}^i = f(x_t^i, u_t^i, \theta_t^i)\}$$

In other words, it is the set of all possible outcomes θ_t^i that could be selected by Nature such that, when applied to the transition function f , they lead to a valid next state x_{t+1}^i .

1.3 Bellman Equation

Let $J^*(x_t^i)$ be the cost of reaching the goal from state x_t^i at time t . Let $l(x_t^i, u_t^i, \theta_t^i)$ be the one-step cost incurred at state x_t^i , after taking action u_t^i and facing Nature's action θ_t^i . If we pick any state x_t^i , value function can be expressed as:

$$J^*(x_t^i) = \min_{u_t^i} \mathbb{E}_{\theta_t^i} [l(x_t^i, u_t^i, \theta_t^i) + J^*(x_{t+1}^i)]$$

This takes into account both the immediate cost and the expected cost-to-go from the next state, under the transition dynamics.

$$J^*(x_t^i) = \min_{u_t^i} \mathbb{E}_{\theta_t^i} [l(x_t^i, u_t^i, \theta_t^i) + J^*(x_{t+1}^i)]$$

Assuming l does not depend on x_{t+1}^i directly, we can rewrite this as:

$$= \min_{u_t^i} \sum_{x_{t+1}^i} [l(x_t^i, u_t^i, \theta_t^i) + J^*(x_{t+1}^i)] \cdot P(x_{t+1}^i \mid x_t^i, u_t^i)$$

To put it simply, for each action u_t^i , compute the expected total cost—comprising the immediate cost $l(x_t^i, u_t^i, \theta_t^i)$ and the future cost-to-go $J^*(x_{t+1}^i)$ —weighted by the probability of reaching each possible next state x_{t+1}^i . We then choose the action that minimizes this expected cost.

1.4 Value and Policy Iteration

Value Iteration

Value Iteration is an iterative algorithm that computes the optimal value function J^* by repeatedly applying the Bellman update. It works by sweeping over all states and updating their cost-to-go estimates based on the best expected cost achievable from that state. Once J^* converges, an optimal policy can be extracted by acting greedily with respect to it.

$$J^*(x_t^i) = \min_{u_t^i} \left[l(x_t^i, u_t^i) + \sum_{x_{t+1}^i} J^*(x_{t+1}^i) \cdot P(x_{t+1}^i \mid x_t^i, u_t^i) \right]$$

Algorithm 1 Value Iteration

```

1: Input: States, Actions, transition probabilities  $P$ , reward function  $R$ , discount factor  $\gamma$ ,
   threshold  $\theta$ 
2: for each state  $s$  in States do
3:    $V(s) \leftarrow 0$ 
4: end for
5: repeat
6:    $\delta \leftarrow 0$ 
7:   for each state  $s$  in States do
8:      $v_{\text{old}} \leftarrow V(s)$ 
9:      $V(s) \leftarrow \max_a \sum_{s'} P(s, a, s') [R(s, a, s') + \gamma V(s')]$ 
10:     $\delta \leftarrow \max(\delta, |v_{\text{old}} - V(s)|)$ 
11:   end for
12: until  $\delta < \theta$ 
13: return  $V$ 

```

Policy Iteration

Policy Iteration alternates between two steps: (1) evaluating the current policy to compute its value function, and (2) improving the policy by acting greedily with respect to this value function. Unlike Value Iteration, it searches directly in the space of policies.

Policy Evaluation:

$$J^\pi(x_t^i) = l(x_t^i, \pi(x_t^i)) + \sum_{x_{t+1}^i} J^\pi(x_{t+1}^i) \cdot P(x_{t+1}^i \mid x_t^i, \pi(x_t^i))$$

Policy Improvement:

$$\pi_{\text{new}}(x_t^i) = \arg \min_{u_t^i} \left[l(x_t^i, u_t^i) + \sum_{x_{t+1}^i} J^\pi(x_{t+1}^i) \cdot P(x_{t+1}^i | x_t^i, u_t^i) \right]$$

If π_{new} improves the value for at least one state, repeat the steps with the updated policy.

Algorithm 2 Policy Iteration

```

1: Input: States, Actions,  $P$ ,  $R$ ,  $\gamma$ ,  $\theta$ 
2: for each state  $s$  in States do
3:   Initialize policy  $\pi(s)$  arbitrarily
4: end for
5: repeat
6:    $V \leftarrow \text{PolicyEvaluation}(\pi, \text{States}, P, R, \gamma, \theta)$ 
7:   policy_stable  $\leftarrow \text{True}$ 
8:   for each state  $s$  in States do
9:      $old\_action \leftarrow \pi(s)$ 
10:     $\pi(s) \leftarrow \arg \max_a \sum_{s'} P(s, a, s') [R(s, a, s') + \gamma V(s')]$ 
11:    if  $old\_action \neq \pi(s)$  then
12:      policy_stable  $\leftarrow \text{False}$ 
13:    end if
14:  end for
15: until policy_stable
16: return  $\pi$ ,  $V$ 

```

Algorithm 3 Policy Evaluation

```

1: Input: Policy  $\pi$ , States,  $P$ ,  $R$ ,  $\gamma$ ,  $\theta$ 
2: for each state  $s$  in States do
3:    $V(s) \leftarrow 0$ 
4: end for
5: repeat
6:    $\delta \leftarrow 0$ 
7:   for each state  $s$  in States do
8:      $v_{\text{old}} \leftarrow V(s)$ 
9:      $V(s) \leftarrow \sum_{s'} P(s, \pi(s), s') [R(s, \pi(s), s') + \gamma V(s')]$ 
10:     $\delta \leftarrow \max(\delta, |v_{\text{old}} - V(s)|)$ 
11:   end for
12: until  $\delta < \theta$ 
13: return  $V$ 

```

1.5 Reinforcement Learning



Figure 2: The success of the AlphaGo series marks the maturation of the single-agent decision-making process. The year 2019 was a booming year for MARL techniques; remarkable progress was achieved in solving immensely challenging multi-player real-strategy video games and multi-player incomplete-information poker games.

Reinforcement Learning is mainly used in when the agent learns through interaction with the environment over time. RL is typically applied to infinite-horizon, probabilistic control problems. This contrasts with the finite-horizon dynamic programming (DP) problems with stochastic transitions. The RL framework combines two core tasks:

- Learning the transition dynamics

$$P(x_{t+1}^i, \theta_t^i \mid x_t^i, u_t^i)$$

through exploration and data collection,

- Computing the optimal policy that minimizes long-term cost using methods like DP.

Phases of a Model-Based RL Approach

- (i) Learning Phase: The agent explores the environment to estimate the transition probabilities

$$P(x_{t+1}^i \mid x_t^i, u_t^i)$$

by visiting various states and recording outcomes of different actions. Once enough data is collected, the system dynamics are treated as known.

- (ii) Planning Phase: With the estimated model in hand, the agent computes an optimal policy using dynamic programming methods such as Value Iteration (VI) or Policy Iteration (PI).
- (iii) Execution Phase: The agent follows the computed policy to act in the environment and accomplish its objective.

Chapter 2

Day 2

In model-free RL, we do not need to explicitly model Nature's actions θ_t^i . Instead, we directly learn the transition probabilities:

$$P(x_{t+1}^i \mid x_t^i, u_t^i)$$

Suppose you are given a policy $\pi(x)$, which maps states to actions. Given a state x_t^i , you know the action $u_t^i = \pi(x_t^i)$ to take. You can evaluate this policy by simulating its performance. Suppose you generate a sequence of incurred costs $c_1, c_2, \dots, c_m$ over m trials or episodes. These costs can be averaged incrementally using the recurrence:

$$m_i = \frac{(i-1)}{i} m_{i-1} + \frac{1}{i} c_i = \frac{1}{i} \sum_{j=1}^i c_j$$

This approach allows you to estimate the expected cost without storing all previous samples.

$$J^\pi(x_t^i) = l(x_t^i, u_t^i) + \sum_{x_{t+1}^i} J^\pi(x_{t+1}^i) \cdot P(x_{t+1}^i \mid x_t^i, u_t^i)$$

This represents the expected total cost of starting in state x_t^i , taking the action $u_t^i = \pi(x_t^i)$, incurring immediate cost $l(x_t^i, u_t^i)$, and then continuing to follow policy π from the next state onward. The second term computes the expected future cost by summing over all possible next states, weighted by their transition probabilities.

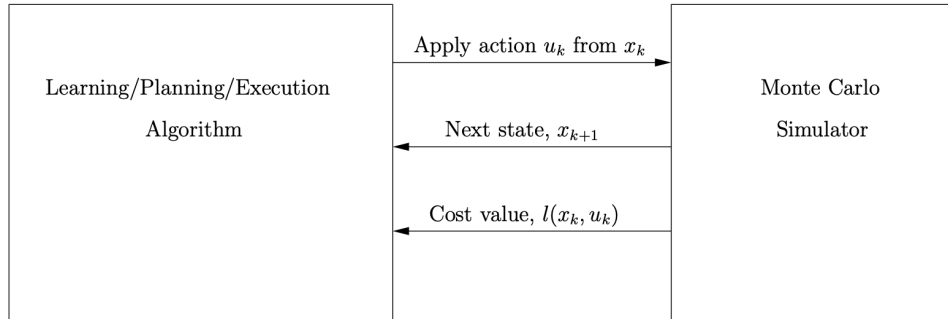


Figure 10.11: The general framework for reinforcement learning (or simulation-based dynamic programming).

When the transition model is unknown, we can use a simulator to estimate the value function. Suppose we run N simulated episodes. In each run, the simulator selects an action u_t^i , transitions to a next state x_{t+1}^i , and records an observed cost $l(x_t^i, u_t^i)$. We can update the estimated value function incrementally across episodes. After i simulations, the update looks like:

$$\hat{J}_\pi(x_t^i) \leftarrow \hat{J}_\pi(x_t^i) + \frac{1}{i} \left[\sum_{t=0}^T l(x_t^i, u_t^i) - \hat{J}_\pi(x_t^i) \right]$$

Notice that this update rule resembles the recursive mean formula:

$$m_i = m_{i-1} + \frac{1}{i} (c_i - m_{i-1})$$

where c_i is the total cost observed in the i -th simulation.

2.1 Temporal Difference (TD) Learning

Instead of waiting until the end of an episode to update value estimates, TD methods perform updates incrementally—immediately after observing a single transition and reward. For a given transition tuple $(x_t^i, u_t^i, x_{t+1}^i)$, we can define the TD update as:

$$\hat{J}_\pi(x_t^i) \leftarrow \hat{J}_\pi(x_t^i) + \alpha \left(l(x_t^i, u_t^i) + \hat{J}_\pi(x_{t+1}^i) - \hat{J}_\pi(x_t^i) \right)$$

This update uses the temporal difference:

$$\delta_t = l(x_t^i, u_t^i) + \hat{J}_\pi(x_{t+1}^i) - \hat{J}_\pi(x_t^i)$$

Here, the action u_t^i is selected using the current policy:

$$u_t^i = \pi(x_t^i)$$

As each state x^i is visited during simulation, we update its estimated value in-place using the observed TD error. For example, if reaching x_2^i yields an error term d_1 , we update:

$$\hat{J}_\pi(x_1^i) \leftarrow \hat{J}_\pi(x_1^i) + \frac{d_1}{V_1}$$

where V_1 tracks how many times x_1^i has been visited.

If the next state x_3^i is then reached, we compute a second TD error d_2 , and the updated rule becomes:

$$\hat{J}_\pi(x_1^i) \leftarrow \hat{J}_\pi(x_1^i) + \frac{d_1}{V_1} + \frac{d_2}{V_2}$$

The idea is to propagate the TD error back along the chain of visited states, weighted by the inverse of their visitation counts.

This yields:

$$\hat{J}_\pi(x_1^i) \leftarrow \hat{J}_\pi(x_1^i) + \left[l(x_1^i, u_1^i) + l(x_2^i, u_2^i) + \hat{J}_\pi(x_3^i) - \hat{J}_\pi(x_1^i) \right]$$

We can generalize this update rule as:

$$\hat{J}_\pi(x_0^i) = \hat{J}_\pi(x_0^i) + \left[l(x_t^i, u_t^i) + l(x_{t+1}^i, u_{t+1}^i) + \cdots + \hat{J}_\pi(x_j^i) - \hat{J}_\pi(x_t^i) \right]$$

where the correction accumulates costs and bootstraps from the final value.

TD(λ) allows us to interpolate between Monte Carlo and one-step TD learning using a trace decay parameter $\lambda \in [0, 1]$. This method accelerates convergence by distributing the TD error across past visited states proportionally to how recently they were visited.

When a TD error d_t is observed at time t , the value function is updated as:

$$\begin{aligned} \hat{J}_\pi(x_1^i) &= \hat{J}_\pi(x_1^i) + \lambda^{t-1} \frac{d_t}{V_1} \\ \hat{J}_\pi(x_2^i) &= \hat{J}_\pi(x_2^i) + \lambda^{t-2} \frac{d_t}{V_2} \\ &\dots \\ \hat{J}_\pi(x_t^i) &= \hat{J}_\pi(x_t^i) + \frac{d_t}{V_t} \end{aligned}$$

When $\lambda = 1$, TD(λ) behaves like the full-return Monte Carlo method—it propagates error back to all visited states. When $\lambda = 0$, we recover the original one-step TD algorithm, which only updates the immediately preceding state. So the generalized TD(λ) update becomes:

$$\hat{J}_\pi(x_0^i) = \hat{J}_\pi(x_0^i) + \frac{1}{V_t} \left[l(x_t^i, u_t^i) + \hat{J}_\pi(x_{t+1}^i) - \hat{J}_\pi(x_t^i) \right]$$

Algorithm 4 TD(0) Policy Evaluation

```

1: Input: Policy  $\pi$ , States, Actions, Simulator, Learning rate schedule  $\{\alpha_t\}$ , Discount
   factor  $\gamma$ , Number of episodes  $N$ 
2: for each state  $s$  in States do
3:   Initialize  $V(s) \leftarrow 0$ 
4:   Initialize  $visit\_count(s) \leftarrow 0$ 
5: end for
6: for episode = 1 to  $N$  do
7:   Initialize  $s \leftarrow$  starting state
8:   while  $s$  is not terminal do
9:      $a \leftarrow \pi(s)$ 
10:    Simulate taking action  $a$  in state  $s$ , observe next state  $s'$  and cost  $l(s, a)$ 
11:     $visit\_count(s) \leftarrow visit\_count(s) + 1$ 
12:     $\alpha \leftarrow \alpha_{visit\_count(s)}$ 
13:     $V(s) \leftarrow V(s) + \alpha \left( l(s, a) + \gamma V(s') - V(s) \right)$ 
14:     $s \leftarrow s'$ 
15:   end while
16: end for
17: return  $V$ 

```

2.2 Q-learning

Q-learning tries to compute an optimal policy directly without requiring a model of the environment or a fixed policy. The object in Q-learning is the Q-function:

$$Q^*(x_t^i, u_t^i)$$

This function extends the value function $J^*(x_t^i)$ by associating costs not only with states but with specific state-action pairs. The Q-value $Q^*(x_t^i, u_t^i)$ represents the expected cumulative cost incurred by starting in state x_t^i , taking action u_t^i , and then following the optimal policy thereafter.

Formally, this can be written as:

$$Q^*(x_t^i, u_t^i) = l(x_t^i, u_t^i) + \mathbb{E}_{x_{t+1}^i} \left[\min_{u'} Q^*(x_{t+1}^i, u') \right]$$

If the action u_t^i is the optimal one at state x_t^i , then its Q-value is equal to the optimal value function:

$$Q^*(x_t^i, u_t^i) = J^*(x_t^i)$$

One useful interpretation of Q-values is that they quantify the cost of taking a particular action now—whether optimal or not—followed by behaving optimally in the future. Q-values allow for reasoning about decision quality at the action level, enabling both off-policy learning and policy improvement.

Q-learning can be generalized to interpolate between relying entirely on the current value estimate versus fully trusting the observed transition. This is done by introducing a parameter $\rho \in [0, 1]$ that weighs the influence of the newly observed estimate. The general TD(0) update can be written as:

$$\text{TD}(\rho) = \hat{J}_\pi(x_t^i) = (1 - \rho)\hat{J}_\pi(x_t^i) + \rho \left[l(x_t^i, u_t^i) + \hat{J}_\pi(x_{t+1}^i) \right]$$

This blends the old estimate with the new sample-based estimate. The weighting ρ reflects the confidence in the current sample: transitions that occur more frequently are more likely to be sampled and are updated more often. For Q-learning, the Q-function update becomes:

$$\hat{Q}^*(x_t^i, u_t^i) = (1 - \rho)\hat{Q}^*(x_t^i, u_t^i) + \rho \left(l(x_t^i, u_t^i) + \min_{u'} \hat{Q}^*(x_{t+1}^i, u') \right)$$

Once Q^* has converged, the optimal value function can be recovered by minimizing over actions:

$$J^*(x_t^i) = \min_{u_t^i \in \mathcal{U}(x_t^i)} Q^*(x_t^i, u_t^i)$$

The best cost-to-go from a state is simply the minimum Q-value across all available actions from that state.

The full Bellman-optimality form of the Q-function is:

$$Q^*(x_t^i, u_t^i) = l(x_t^i, u_t^i) + \sum_{x_{t+1}^i} P(x_{t+1}^i \mid x_t^i, u_t^i) \cdot \min_{u'} Q^*(x_{t+1}^i, u')$$

This shows that the optimal Q-value is the immediate cost plus the expected cost-to-go, where the future cost is minimized over all possible next actions.

The value function for an optimal policy can be written as:

$$J^*(x_t^i) = \min_{u_t^i} \left[l(x_t^i, u_t^i) + \sum_{x_{t+1}^i} P(x_{t+1}^i \mid x_t^i, u_t^i) \cdot J^*(x_{t+1}^i) \right]$$

This expression assumes that we know both the immediate cost $l(x_t^i, u_t^i)$ and the cost-to-go from the next state $J^*(x_{t+1}^i)$ ahead of time. However, in practice, we only get this information when we actually visit the state and simulate the outcome.

To compute $J^*(x_{t+1}^i)$, we would also need to know which action the agent would take optimally at the next step. This is not known in advance.

Thus, we shift the minimization to occur within the Q-function, allowing us to decouple the decision-making process:

$$J^*(x_t^i) = \min_{u_t^i} Q^*(x_t^i, u_t^i)$$

The Q-function lets one represent this optimization as a two-step process: taking an action and then acting optimally afterward. This avoids having to know the entire value function structure in advance.

We can use the learned Q-function to improve the current policy by selecting, at each state, the action with the lowest estimated cost.

1. $Q^*(x_t^i, u_t^i) = l(x_t^i, u_t^i) + \sum_{x_{t+1}^i} P(x_{t+1}^i \mid x_t^i, u_t^i) \cdot J^*(x_{t+1}^i)$
2. $\pi_{\text{new}}(x_t^i) = \arg \min_{u_t^i} Q^*(x_t^i, u_t^i)$

The simulator is used to estimate $l(x_t^i, u_t^i)$ and $P(x_{t+1}^i \mid x_t^i, u_t^i)$ through sampled transitions. Once the Q-values converge, we extract the optimal policy by choosing the action that minimizes Q^* at each state.

Algorithm 5 Q-Learning

```

1: Input: States, Actions, Simulator, Learning rate schedule  $\{\alpha_t\}$ , Discount factor  $\gamma$ , Num-
   ber of episodes  $N$ , Exploration schedule (e.g.,  $\varepsilon$ -greedy)
2: for each  $(x, u)$  pair do
3:   Initialize  $Q(x, u) \leftarrow 0$ 
4: end for
5: for episode = 1 to  $N$  do
6:   Initialize  $x \leftarrow$  starting state
7:   while  $x$  is not terminal do
8:     Choose action  $u$  using an exploration strategy (e.g.,  $\varepsilon$ -greedy w.r.t.  $Q$ )
9:     Simulate taking action  $u$  in state  $x$  to get:
10:    next state  $x'$ , and cost  $l(x, u)$ 
11:     $\alpha \leftarrow \alpha_t$  (based on current visitation or schedule)
12:    Update Q-value:

```

$$Q(x, u) \leftarrow Q(x, u) + \alpha \left(l(x, u) + \gamma \min_{u'} Q(x', u') - Q(x, u) \right)$$

```

13:     $x \leftarrow x'$ 
14:  end while
15: end for
16: Return: Optimal policy  $\pi^*$  where

```

$$\pi^*(x) = \arg \min_u Q(x, u)$$

2.3 Deep Reinforcement Learning (Deep RL)

We start thinking of deep-learnign when the state space $\mathcal{X}$ becomes very large—or even continuous—making it infeasible to store or update value functions (or Q-functions) in a table. Instead of storing $J^\pi(x_t^i)$ or $Q^\pi(x_t^i, u_t^i)$ explicitly for every state or state-action pair, we approximate them using parameterized function approximators such as neural networks.

$$\begin{aligned} \hat{J}_\pi(x_t^i; \omega) &\approx J^\pi(x_t^i) \\ \hat{Q}_\pi(x_t^i, u_t^i; \omega) &\approx Q^\pi(x_t^i, u_t^i) \end{aligned}$$

Here, ω denotes the parameters (e.g., neural network weights), and the approximation is learned over time. Since the number of parameters $|\omega|$ can be much smaller than the size of the state space $|\mathcal{X}|$, this method allows RL to scale to high-dimensional environments. We update the parameters ω using standard RL techniques like Temporal Difference (TD) Learning or Monte Carlo (MC) Estimation

We can use gradient descent to minimize the discrepancy between the approximate value estimates and their targets.

2.4 Function Approximation in RL

When the state space becomes large or continuous, storing exact value functions becomes infeasible. Instead, we approximate value functions using parameterized models.

Types of Function Approximators

1. **Linear combinations of features**
2. **Neural networks**

We want to find parameters ω that minimize the discrepancy between the true value function $J^\pi(x_t^i)$ and the approximate prediction $\hat{J}_\pi(x_t^i; \omega)$:

$$\min_{\omega} \mathbb{E}_{\mathcal{T}} \left[\left(J^\pi(x_t^i) - \hat{J}_\pi(x_t^i; \omega) \right)^2 \right]$$

To perform gradient descent, we need the parameter update $\Delta\omega$:

$$\Delta\omega = -\frac{1}{2} \nabla_{\omega} L(\omega)$$

This leads to the update:

$$\Delta\omega = \alpha \cdot \mathbb{E}_{\mathcal{T}} \left[\left(J^\pi(x_t^i) - \hat{J}_\pi(x_t^i; \omega) \right) \cdot \nabla_{\omega} \hat{J}_\pi(x_t^i; \omega) \right]$$

Here:

- The first term is the error between the true and predicted values (oracle minus approximation),
- The second term is the gradient of the function approximator with respect to ω .

Each state x_t^i is represented by a feature vector:

$$f(x_t^i) = \begin{bmatrix} f_1(x_t^i) \\ f_2(x_t^i) \\ \vdots \\ f_n(x_t^i) \end{bmatrix}$$

We then approximate the value function as a linear combination of these features:

$$\hat{J}_\pi(x_t^i; \omega) = f(x_t^i)^\top \omega = \sum_j f_j(x_t^i) \cdot \omega_j$$

The corresponding loss is quadratic in ω :

$$L(\omega) = \mathbb{E}_{\mathcal{T}} \left[\left(J^\pi(x_t^i) - f(x_t^i)^\top \omega \right)^2 \right]$$

This setup enables the use of stochastic gradient descent, which is known to converge under appropriate conditions. The parameter update is:

$$\text{Update} = \text{stepsize} \times \text{error} \times \text{feature values}$$

The target value $J^\pi(x_t^i)$ is not known in advance and must be estimated using either simulation or experience data.

Different learning methods define the target differently:

- Monte Carlo (MC): Use full return $J^\pi(x_t^i)$ from trajectory
- TD(0): Use single-step bootstrapped target:

$$\text{Target} = l(x_t^i, u_t^i) + \gamma \hat{J}_\pi(x_{t+1}^i; \omega)$$

- TD(λ): Use λ -return as the target

2.5 Training Value Function Approximators

Suppose we treat value function learning as a supervised learning problem. Then we can form training pairs consisting of states and target values.

If we somehow had access to the true value function $J^\pi(x_t^i)$ (e.g., from an oracle), we could directly train a function approximator by minimizing prediction error:

$$\Delta\omega = \alpha \cdot \left(J^\pi(x_t^i) - \hat{J}(x_t^i; \omega) \right) \cdot \nabla_\omega \hat{J}(x_t^i; \omega)$$

This is the standard supervised learning update, where the loss is computed against the true label.

More realistically, we estimate the target using RL-based targets, such as:

$$J^\pi(x_t^i) \approx l(x_t^i, u_t^i) + \gamma \hat{J}(x_{t+1}^i; \omega)$$

Then the update becomes:

$$\Delta\omega = \alpha \cdot \left(l(x_t^i, u_t^i) + \gamma \hat{J}(x_{t+1}^i; \omega) - \hat{J}(x_t^i; \omega) \right) \cdot \nabla_\omega \hat{J}(x_t^i; \omega)$$

This is a bootstrapped, biased update typical in TD-learning. All of this can also be extended to Q-values:

$$\hat{Q}(x_t^i, u_t^i; \omega)$$

Let:

$$\mathcal{D} = \{x_t^i, J_\pi(x_t^i)\}$$

We collect transitions and value estimates under a policy and store them as a dataset. This concept forms the basis for **experience replay**, where transitions are reused for training rather than discarded after a single use.

Algorithm 6 Deep Q-Network (DQN)

- 1: **Input:** Learning rate α , discount factor γ , exploration schedule ε , replay buffer $\mathcal{D}$, batch size B
- 2: Initialize Q-network $\hat{Q}(x, u; \omega)$ with random weights ω
- 3: Initialize target network $\hat{Q}^-(x, u; \omega^-)$ with weights $\omega^- \leftarrow \omega$
- 4: **for** episode = 1 to N **do**
- 5: Initialize state x
- 6: **while** not terminal **do**
- 7: Select action u using ε -greedy policy from $\hat{Q}(x, u; \omega)$
- 8: Execute action u , observe cost $l(x, u)$ and next state x'
- 9: Store transition $(x, u, l(x, u), x')$ in replay buffer $\mathcal{D}$
- 10: Sample minibatch $\{(x_j, u_j, l_j, x'_j)\}_{j=1}^B$ from $\mathcal{D}$
- 11: **for** each sampled transition **do**
- 12: $y_j \leftarrow l_j + \gamma \min_{u'} \hat{Q}^-(x'_j, u'; \omega^-)$
- 13: **end for**
- 14: Update ω by minimizing the loss:

$$L(\omega) = \frac{1}{B} \sum_{j=1}^B \left(y_j - \hat{Q}(x_j, u_j; \omega) \right)^2$$

- 15: Optionally update target network: $\omega^- \leftarrow \omega$ (periodically)
 - 16: $x \leftarrow x'$
 - 17: **end while**
 - 18: **end for**
 - 19: **return** $\hat{Q}(x, u; \omega)$
-

Chapter 3

Additional Materials

3.1 Problem Formulation: Stochastic Games

A stochastic game is a generalization of the MDP to the multi-agent setting. It is defined by the tuple:

$$\langle N, \mathbb{S}, \{\mathbb{A}^i\}_{i=1}^N, P, \{R^i\}_{i=1}^N, \gamma \rangle$$

Where:

- N : number of agents. When $N = 1$, SG reduces to an MDP; when $N \gg 2$, we refer to many-agent settings.
- $\mathbb{S}$: shared state space among all agents.
- $\mathbb{A}^i$: action space for agent i ; the joint action space is $\mathbb{A} = \mathbb{A}^1 \times \dots \times \mathbb{A}^N$.
- $P(s' \mid s, \mathbf{a})$: transition probability function; describes how the environment transitions from s to s' given joint action $\mathbf{a}$.
- $R^i(s, \mathbf{a}, s')$: reward function for agent i , giving a scalar for the transition $(s, \mathbf{a}, s')$.
- $\gamma \in [0, 1]$: discount factor for future rewards.

Each agent i has a policy $\pi^i \in \Pi^i$, mapping states to a distribution over actions. At each time step, all agents act simultaneously. The next state is sampled as $s_{t+1} \sim P(\cdot \mid s_t, \mathbf{a}_t)$, and each agent receives its reward $R^i(s_t, \mathbf{a}_t, s_{t+1})$. Notice this is a bit different from the taxonomy we have covered in the first class.

Value Function (we have seen this before!):

$$V^{\pi^i, \pi^{-i}}(s) = \mathbb{E} \left[\sum_{t \geq 0} \gamma^t R^i(s_t, \mathbf{a}_t, s_{t+1}) \mid a_t^i \sim \pi^i(\cdot \mid s_t) \right]$$

This highlights that each agent's return depends not only on its own policy but also on the policies of all others.

3.2 Solving Stochastic Games

Stochastic games can be interpreted as a sequence of normal-form (matrix) games. At each time step t , the stage game defines the payoff matrix for the current state s_t . Rows/columns of this matrix correspond to actions of agents, and entries reflect rewards for joint actions.

Example: In driving through an intersection, if both agents act selfishly and rush, they may crash (suboptimal). Strategic reasoning is needed to coordinate for better outcomes.

Tractable subclasses of SGs

- Single-controller SG: Transitions depend only on one agent's action.
- SR-SIT SG (Separable reward, State-Independent Transition):
 - $R^i(s, a) = \alpha(s) + \beta(a)$, for some functions α, β
 - $P(s' | s, a) = P(s' | a)$: transition depends only on actions

3.3 Value-Based MARL Methods

The Q-learning update generalizes to multi-agent settings. For agent i , given sampled transitions $(s_t, \mathbf{a}_t, R^i, s_{t+1})$, we update:

$$Q^i(s_t, \mathbf{a}_t) \leftarrow Q^i(s_t, \mathbf{a}_t) + \alpha (R^i + \gamma \cdot \text{eval}^i(\{Q^j(s_{t+1}, \cdot)\}_{j=1}^N) - Q^i(s_t, \mathbf{a}_t))$$

Here, the standard max operator is replaced by an evaluation operator eval^i that considers all agents' Q-values.

$$\text{eval}^i(\{Q^j\}_{j=1}^N) = V^i(s_{t+1}, \{\text{solve}^j(\{Q^k\}_{k=1}^N)\}_{j=1}^N)$$

The solve^j function determines the optimal policy for each agent at equilibrium. Note that the algorithms below are similar to their single-agent counterparts that we have already gone through in the previous chapter

3.4 Policy-Based MARL Methods

Value-based methods suffer from scalability issues due to the joint action space. Policy-based methods instead learn a parameterized policy $\pi_{\theta^i}^i$ for each agent.

Let $\theta = (\theta^1, \dots, \theta^N)$ and define the joint policy as $\pi_\theta = \prod_{i=1}^N \pi_{\theta^i}^i$. The objective for agent i is:

$$J^i(\theta) = \mathbb{E}_{s, \mathbf{a} \sim \pi_\theta} \left[\sum_{t \geq 0} \gamma^t R_t^i \right]$$

Policy gradient:

$$\nabla_{\theta^i} J^i(\theta) = \mathbb{E}_{s, \mathbf{a} \sim \pi_\theta} [\nabla_{\theta^i} \log \pi_{\theta^i}^i(a^i | s) \cdot Q^i(s, \mathbf{a})]$$

This is used in MADDPG, where each agent learns its own deterministic or stochastic policy.

3.5 Solution: Nash Equilibrium

A Nash equilibrium (NE) is a joint policy where no agent can benefit by deviating unilaterally:

$$\pi^{i,*} \in \text{Br}(\pi^{-i}) = \left\{ \arg \max_{\pi^i} \mathbb{E}_{\pi^i, \pi^{-i}} [R^i(a^i, a^{-i})] \right\}$$

A stronger version of NE, where each policy depends only on the current state and satisfies:

$$V^{\pi^{i,*}, \pi^{-i,*}}(s) \geq V^{\pi^i, \pi^{-i,*}}(s), \quad \forall s, \forall \pi^i$$

We incorporate NE directly into the Q-learning update by replacing the evaluation operator with a Nash-based operator:

$$\text{eval}_{\text{Nash}}^i(\{Q^j(s_{t+1}, \cdot)\}_j) = V^i(s_{t+1}, \{\text{Nash}^j(\{Q^k\}_k)\}_j)$$

Nash Q-learning update:

$$Q^i(s, \mathbf{a}) \leftarrow \sum_{s'} P(s' | s, \mathbf{a}) [R^i(s, \mathbf{a}, s') + \gamma \cdot \text{eval}_{\text{Nash}}^i(\{Q^j(s', \cdot)\}_j)]$$

Finding NE in 2-player general-sum games reduces to solving a linear complementarity problem (LCP). Solving NE is:

- PPAD-hard for 2-player general-sum games
- NP-hard for uniqueness checking
- PSPACE-hard for general SGs

This implies that computing exact NE in multi-agent settings is generally intractable for large games.

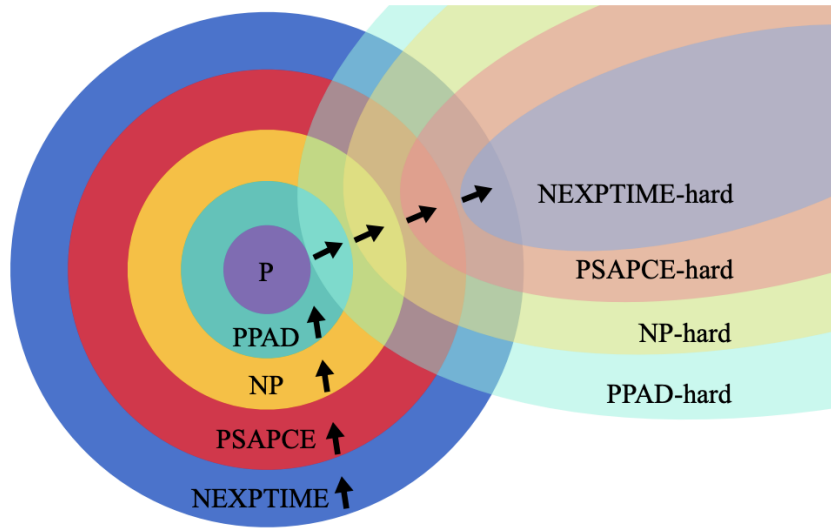


Figure 5: The landscape of different complexity classes. Relevant examples are 1) solving the NE in a two-player zero-sum game, P -complete (Neumann, 1928), 2) solving the NE in a general-sum game, $PPAD$ -hard (Daskalakis et al., 2009), 3) checking the uniqueness of the NE, NP -hard (Conitzer and Sandholm, 2002), 4) checking whether a pure-strategy NE exists in a stochastic game, $PSPACE$ -hard (Conitzer and Sandholm, 2008), and 5) solving Dec-POMDP, $NEXPTIME$ -hard (Bernstein et al., 2002).